THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 01 OF 01

AI Design #73 — Unified Enterprise Agentic Platform

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

01

YOUR STUDY COMPANION

Stage 5 of 6 — Production Evolution

NAVIGATION

Inside this guide

01	1. New requirement: Adaptive investigation
02	2. Revised architecture
03	3. The new tool execution contract
04	4. Failure scenario: The model requests a dangerous action
05	5. Working Stage 5A implementation
06	6. What did we improve?
07	7. Next production requirement
08	Stage 5 — Progress
09	5.1 New requirement
10	5.2 Revised architecture
11	5.3 Tool request contract: no model-provided scope
12	5.4 Controller policy
13	5.5 Model/tool request flow
14	5.6 Failure scenario: model proposes a dangerous action
15	5.7 Failure scenario: missing evidence
16	5.8 Stage 5A implementation structure
17	5.9 Reported Stage 5A verification
18	5.10 Requirement → Change → Check → Trade-off
19	What changed across the completed work?
20	Reusable principles established

21	Glossary and source map
22	Next planned increment (not included)

CHAPTER

1. New requirement: Adaptive investigation

In Stage 4, we built a working investigation simulator for `INC-42`.

Evolution 1: Real model reasoning and safe tool execution Now we will improve it one requirement at a time.

An engineer asks:

> Investigate INC-42. Checkout API latency increased after deployment. Find the likely cause.

In Stage 4, Python always executed the same two tool calls, then generated a deterministic summary.

That works for predictable incidents. However, future investigations may need different evidence depending on the symptoms.

New requirement: Allow an LLM to choose approved diagnostic tools and analyze their results, without granting it operational authority.

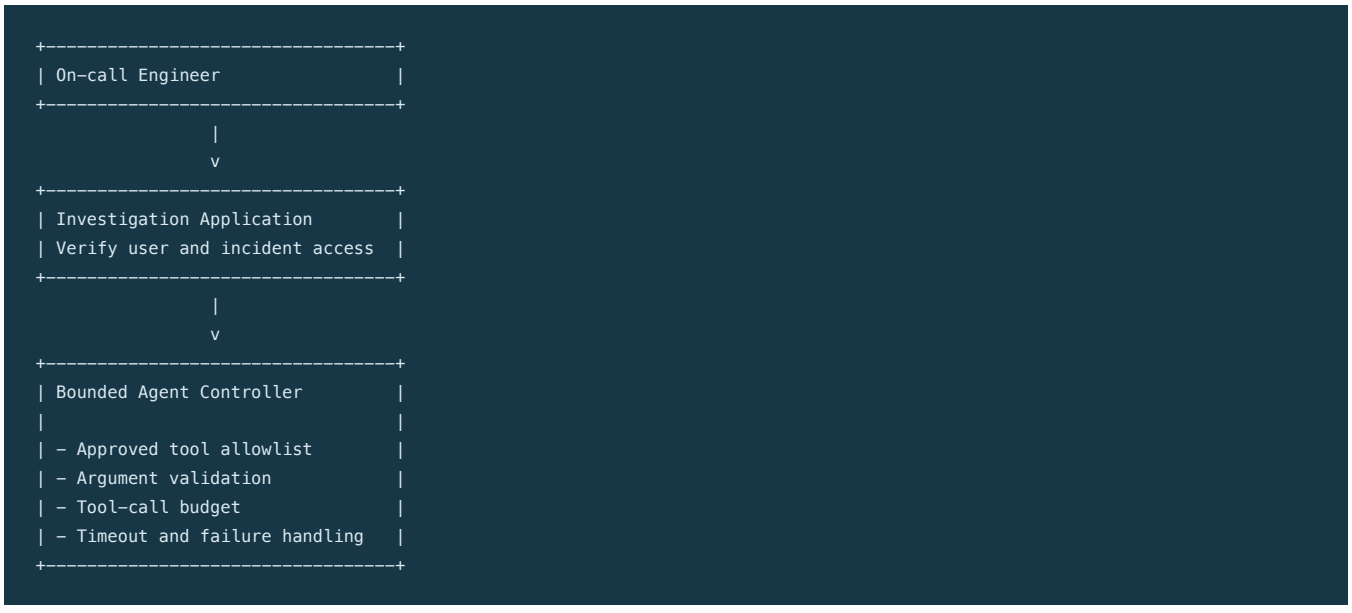
	Stage 4	Stage 5 — Evolution 1
Tool selection	Fixed Python sequence	Model proposes approved tool calls
Analysis	Deterministic mock	Optional real LLM
Tool execution	Application controlled	Still application controlled
Permissions	Synthetic authorization	Same boundaries, with stronger validation
Production changes	Not allowed	Still not allowed

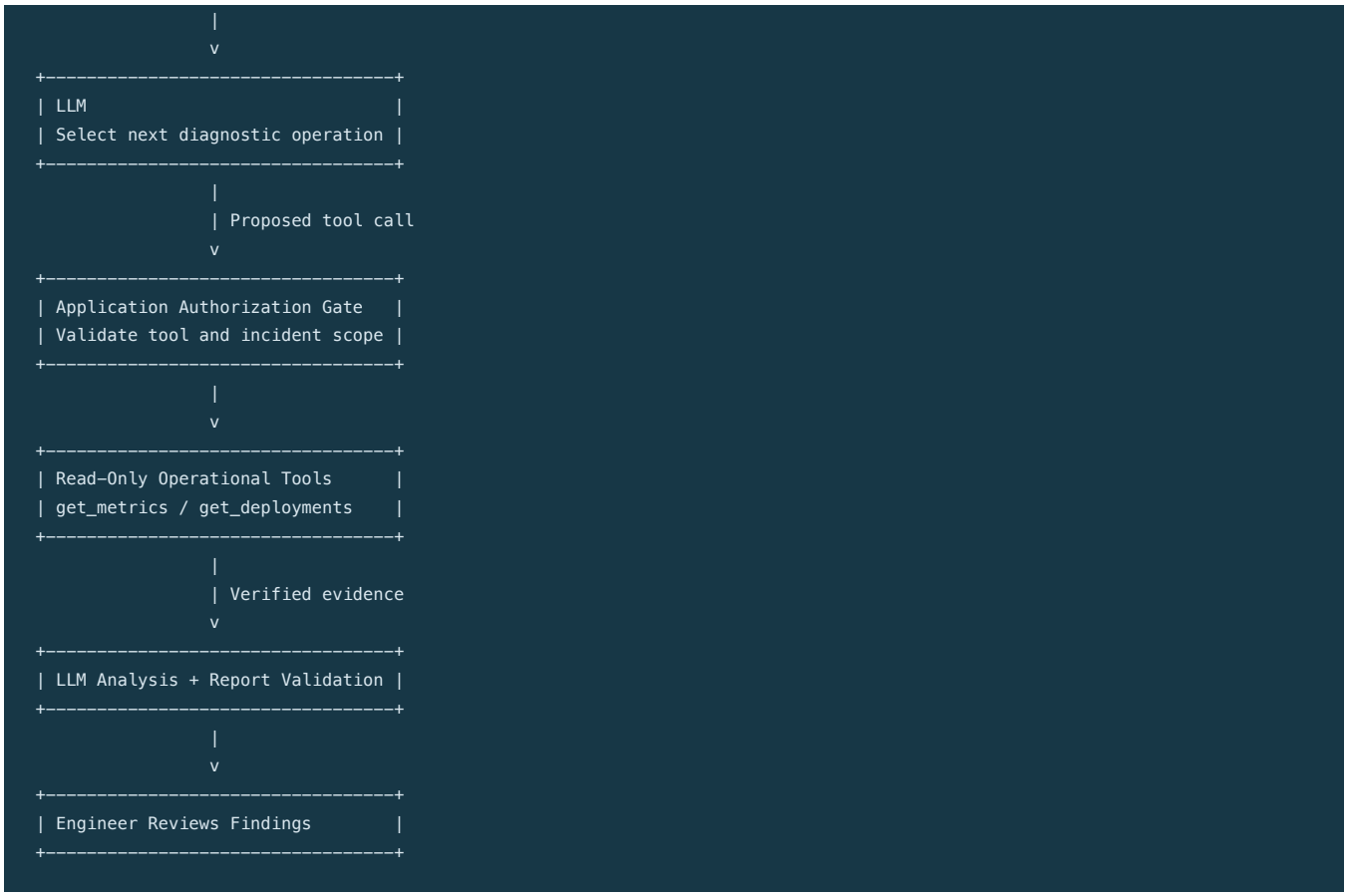
The critical principle remains:

The model decides what to request. The application decides what is permitted and executed.

CHAPTER

2. Revised architecture





The controller manages the loop. It may return evidence to the model for another decision, but it stops when the request is complete or a defined budget is reached.

This follows the official tool-calling pattern: a model returns a function-call request, application code executes it, and the result is passed back to the model.

Important design decision

We will not expose a tool such as `restart_service` or `rollback_deployment` to this investigation agent.

Without those capabilities, the model cannot execute remediation through the approved tool interface, even if it requests it.

This is a stronger boundary than relying only on prompt instructions.

CHAPTER

3. The new tool execution contract

For this increment, we deliberately give both tools zero model-supplied arguments.

```
{
  "name": "get_metrics",
  "arguments": {}
}
```

The server derives the service and tenant from the already-authorized incident.

This prevents the model from choosing an unrelated service by supplying another service ID.

The application also checks the tool name, argument structure, duplicate calls and maximum call count before executing anything.

CHAPTER

4. Failure scenario: The model requests a dangerous action

Suppose the model proposes:

```
{
  "name": "restart_service",
  "arguments": {
    "service": "checkout-api"
  }
}
```

The controller must reject that request.



A correct implementation does not let the model bypass this validation by changing its instructions.

CHAPTER

5. Working Stage 5A implementation

Stage 5A — Bounded Agent Upgrade [23 automated tests passed]

Python source, synthetic fixtures, model adapter, execution controls and automated tests

Read setup instructions

Run locally

After extracting the project:

```
python app_agent.py
```

To simulate missing metrics:

```
python app_agent.py --fail-metrics
```

To run the tests:

```
python -m unittest discover -s tests -v
```

Optional real LLM integration

The package includes `openai_provider.py`, an optional adapter using the OpenAI Responses API.

It supports a tool-proposal call, followed by a structured analysis call using the tool results. The tool-call response format and structured-output approach were checked against official documentation.

Configure it using environment variables:

```
export OPENAI_API_KEY="your-api-key"
export OPENAI_MODEL="your-supported-model"

python app_agent.py --real-model
```

A supported API model and separate API credentials are required. Calls may incur charges.

Verification boundary: The scripted path and offline adapter tests passed. A live model request was not tested, so model-specific behavior is not yet verified.

CHAPTER

6. What did we improve?

Requirement	Verification	Trade-off
Model proposes read tools	Scripted proposal tests	More flexible than fixed sequence
Unknown tool rejected	`restart_service` negative test	Only approved capabilities available
Incident scope protected	Model-supplied service rejected	Less freedom for the model
Tool calls bounded	Excessive and duplicate calls rejected	Investigation may stop early
Missing evidence handled	Timeout simulation	May produce incomplete results
Citations checked	Fabricated evidence ID rejected	Valid IDs can still support incorrect claims

That last limitation is important: checking that a citation exists does not prove that the cited evidence supports the claim. A production version needs stronger grounding evaluation and factual verification.

The mock identity registry also remains unsuitable for real deployment. Real authentication and authorization are still required.

CHAPTER

7. Next production requirement

We have improved the behavior of one investigation assistant. However, we still have ordinary Python tools tightly coupled to that application.

Imagine three teams:

- IT Operations wants to investigate incidents.
- Security Operations wants to inspect service alerts.
- Platform Engineering wants to inspect deployment health.

All three need some of the same monitoring capabilities.

Duplicating these integrations creates maintenance and governance problems.

This gives us the next architectural question:

Should each AI application own its monitoring integrations, or should we expose approved tools through a shared interface?

That is where we will evaluate MCP against direct APIs.

CHAPTER

Stage 5 — Progress

Evolution 1 — Bounded model tool calling [Local tests passed]

Evolution 2 — Shared tools and MCP [Next]

Later: evidence retrieval, durable workflows, approvals, UI integrations, specialist agents and enterprise operations.

Stage 5 Evolution 1 — Bounded Model-Driven Tool Calling

CHAPTER

5.1 New requirement

Stage 4 always reads the same two sources. A more flexible incident assistant may need to decide which approved diagnostic tool to inspect first.

New requirement: Let a model **propose** approved diagnostic tool calls and analyze their results, without letting it control service scope or perform production writes.

Why the baseline is insufficient

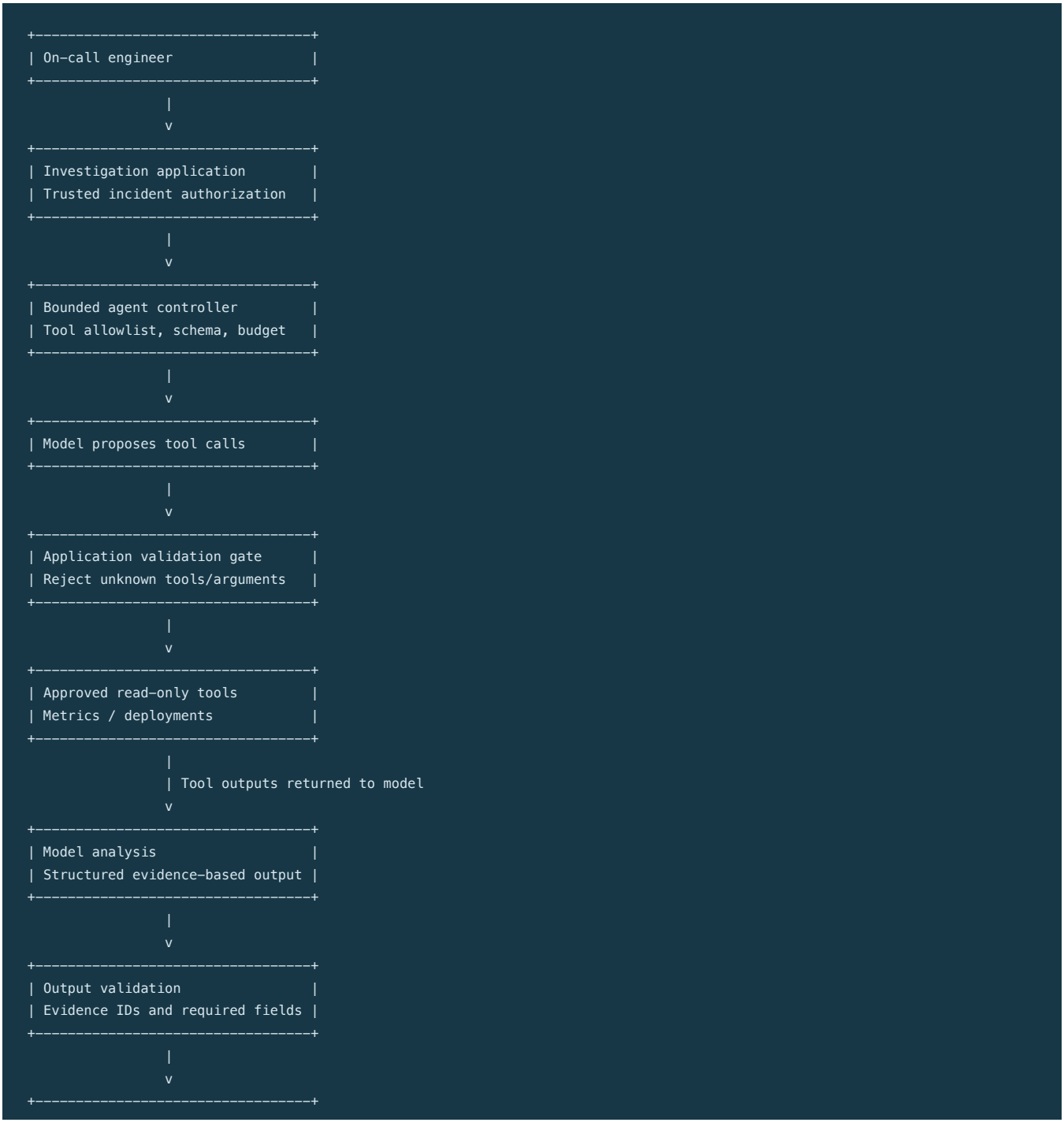
Stage 4 baseline	New need
Python always selects both tools	Model can propose useful tools
Deterministic mock interpretation	Optional real LLM reasoning
Tool execution stays in Python	Must remain in Python

Two known sources	Preserve authorized incident scope
No write tools	Continue to prohibit remediation

This is the **first** Stage 5 increment only. It is not a completed enterprise platform.

CHAPTER

5.2 Revised architecture



```
| Engineer reviews report          |
| No production action             |
+----------------------------------+
```

The important change is **model-driven proposal**, not model-driven authorization.

CHAPTER

5.3 Tool request contract: no model-provided scope

Allowed tool names:

- `get_metrics`
- `get_deployments`

Their model-facing arguments are intentionally empty:

```
{
  "name": "get_metrics",
  "arguments": {}
}
```

Why empty arguments? The controller gets the service and tenant from the incident already authorized by application code. The model cannot override them with `payroll-api`, a different tenant, or a guessed user identity.

Example rejected request:

```
{
  "name": "get_metrics",
  "arguments": {
    "service": "payroll-api"
  }
}
```

The controller rejects this because unexpected arguments are not allowed.

CHAPTER

5.4 Controller policy

Guard	Stage 5 Evolution 1 behavior
Tool allowlist	Only the two named read-only tools
Arguments	Empty JSON object only
Call budget	At most two distinct tool calls per request
Duplicate names / IDs	Reject proposed batch
Authorization	Incident-scoped at API and source-tool boundaries
Missing tool result	Return incomplete-evidence status

Unknown tool	Reject before executing any tool
Model-generated evidence IDs	Validate against retrieved records
Remediation tools	None registered

The proposed batch is validated **before any tool executes**. This prevents partially executing a batch that also contains a prohibited action.

Conceptual controller logic:

```
ALLOWED_TOOLS = {"get_metrics", "get_deployments"}
MAX_TOOL_CALLS = 2


def validate_requests(requests):
    if len(requests) > MAX_TOOL_CALLS:
        raise RejectedToolRequest("TOOL_BUDGET_EXCEEDED")

    seen_names = set()
    seen_ids = set()
    for request in requests:
        if request.name not in ALLOWED_TOOLS:
            raise RejectedToolRequest("UNKNOWN_TOOL")
        if request.arguments != {}:
            raise RejectedToolRequest("UNEXPECTED_ARGUMENTS")
        if request.name in seen_names or request.call_id in seen_ids:
            raise RejectedToolRequest("DUPLICATE_TOOL_REQUEST")
        seen_names.add(request.name)
        seen_ids.add(request.call_id)
```

This excerpt illustrates the contract; the earlier Stage 5A project contained the full implementation and additional input-type checks.

CHAPTER

5.5 Model/tool request flow

The Stage 5A adapter uses **two model phases**:

1. Ask the model to propose up to two approved read-only tool calls.
2. After application-side validation and tool execution, supply matching tool outputs and request a structured investigation report.

An actual tool-call response includes a **call identifier**. The application pairs the result with that identifier when returning the tool output to the model.

Conceptual tool response handed back to the model:

```
{
  "type": "function_call_output",
  "call_id": "call-01",
  "output": "{\"evidence_id\":\"METRIC-07\",\"p95_latency_ms\":1800}"
}
```

The model does **not** directly invoke Python functions. The controller executes the authorized function and supplies its output.

Important limitation: The Stage 5A adapter demonstrated **one proposal phase plus one analysis phase**, not an unlimited autonomous agent loop. A full iterative tool-selection loop was explicitly deferred.

CHAPTER

5.6 Failure scenario: model proposes a dangerous action

The model proposes:

```
{
  "name": "restart_service",
  "arguments": {
    "service": "checkout-api"
  }
}
```

Safe behavior:



Prompt instructions such as “do not restart services” are useful defense-in-depth, but the **absence of a registered write capability** and independent tool validation are the stronger controls.

CHAPTER

5.7 Failure scenario: missing evidence

If the model skips `get_metrics` or monitoring times out:

1. The controller records metrics as missing.
2. Other authorized evidence can still be used.
3. The resulting status is `insufficient_evidence`.
4. A deployment-related hypothesis is not treated as confirmed root cause.
5. No production write is possible.

A denied user is handled earlier: no protected evidence should be retrieved and no unnecessary model request should occur.

CHAPTER

5.8 Stage 5A implementation structure

```
it-ops-assistant-stage5a/
|
+-- app_agent.py
|   Entry point for scripted or optional model mode
|
+-- stage5_agent.py
|   Tool-request validation and guarded dispatcher
|
+-- openai_provider.py
|   Optional hosted model adapter
|
+-- auth.py
+-- tools.py
+-- investigator.py
+-- data/
|   +-- metrics.json
|   +-- deployments.json
|
+-- tests/
    +-- test_investigation.py
    +-- test_stage5_agent.py
    +-- test_openai_adapter_offline.py
```

The default provider, `ScriptedProvider`, simulates a model request but **is not an LLM**.

`OpenAIProvider` was provided as an optional adapter for the OpenAI Responses API using function calls and structured output. It uses an API key and a configured supported model. The adapter's **transport behavior was tested with offline fake responses**, but no real hosted model was called in the lesson.

Local commands from the lab

```
python app_agent.py
python app_agent.py --fail-metrics
python app_agent.py --user operator-9
python -m unittest discover -s tests -v
```

Optional real-model path (requires separate API credentials and a compatible model):

```
export OPENAI_API_KEY="your-api-key"
export OPENAI_MODEL="your-supported-model"
python app_agent.py --real-model
```

Do not put real secrets into study files or source control. The real-model path sends only synthetic incident data in this exercise, may incur API costs, and was **not validated live**.

CHAPTER

5.9 Reported Stage 5A verification

The earlier lesson reported **23 automated tests passing**, including Stage 4 regressions, Stage 5 controller checks, and offline provider-adapter tests. The reported coverage included:

- Successful read-only investigation.
- Rejecting unknown write tool `restart_service`.
- Rejecting model-supplied service arguments.
- Rejecting excessive or duplicate tool calls.
- Handling no tool requests and monitoring timeouts.
- Denying unauthorized service or tenant access.
- Rejecting fabricated evidence IDs.
- Parsing structured model output in an offline stub.
- Pairing `function_call_output` with the correct `call_id`.

Verification boundary: These are previously reported software-test outcomes, not a new test run for this Markdown export. The actual hosted-model request path remains unverified.

CHAPTER

5.10 Requirement → Change → Check → Trade-off

New requirement / risk	Change made	Check used	Trade-off
Need flexible evidence selection	Model proposes tools	Scripted tool-proposal tests	More flexibility; extra model complexity
Model may request unsafe tool	Application allowlist	`restart_service` rejection	Cannot run arbitrary actions
Model may target another service	Scope fixed from authorized incident	Unexpected service-argument rejection	Less model autonomy
Model may loop or over-call	Two distinct tool-call limit	Excess/duplicate tests	Investigation can stop early
Metrics may be unavailable	Partial-result status	Timeout test	Not every investigation gets a conclusion
Model may invent evidence IDs	Citation-ID validator	Fabricated citation test	Does not establish semantic truth
Model/API might be unavailable	Error path rather than an invented answer	Offline adapter and application errors	No guaranteed useful response during provider outage

What is still missing for production?

- Real authentication, delegated identity, and service-level authorization tokens.
- Semantic verification that cited evidence actually supports each claim.
- Persistent, protected audit records and distributed tracing.
- Tool timeouts, retry policies, cancellation, and resource quotas for real infrastructure.
- Version-pinned models, evaluation datasets, rollout gates, and cost budgets.
- Data classification, retention, and privacy review before sending real operational evidence to a hosted model.
- Durable approval workflows if future phases add write actions.
- Shared MCP integration, RAG, AG-UI/A2UI/MCP Apps, or A2A **only when a concrete requirement warrants them**.

Stage 5 Evolution 1 result

The model can **propose** approved evidence reads. The application still owns execution and permissions. Reports are checked for structure and existing evidence IDs. All available operational tools remain **read-only**.

Consolidated Architecture Decisions and Next Step

CHAPTER

What changed across the completed work?

Stage	Main decision	Proven or demonstrated
Stage 1	Investigation copilot, not autonomous operator	Requirements and scope defined
Stage 2	Fixed workflow before adaptive agent	Concepts and trade-offs explained
Stage 3	One application with two guarded reads	Contracts, sequence and failure path defined
Stage 4	Local deterministic Python mock	Synthetic workflow and negative tests reported
Stage 5 Evolution 1	Model proposes tools; app authorizes and executes	Guarded dispatcher and offline model-adapter tests reported

CHAPTER

Reusable principles established

1. **Source of truth:** The monitoring and deployment systems own business facts.
2. **Trust boundary:** The model is not the authorizer and retrieved records are not instructions.
3. **Least privilege:** Initial tools read only the specific incident's authorized service.
4. **Evidence:** Separate observed measurements, hypotheses, and unknowns.
5. **Failure honesty:** Missing evidence must produce an incomplete response, not a confident diagnosis.
6. **Bounded execution:** Tool allowlists, argument schemas, and budgets are application controls.
7. **Incremental complexity:** Add protocols and distributed services only when justified by requirements.

Decisions made through Stage 5A

Decision	Reason	Current result
One incident before many products	Keep scope testable.	INC-42 synthetic case.
Read-only tools	Prevent accidental high-impact actions.	No production write capability.
Source-system ownership	AI text is not authoritative operational data.	Metrics and changes owned by fixtures/source adapters.
Access checks in application and tools	Model/request data are not credentials.	Incident/tenant/service denial tests.
Evidence IDs and uncertainty	A hypothesis needs support and limitations.	Missing-evidence states and citation checks.
Bounded model proposals	Introduce useful flexibility with safe limits.	Two approved, no-argument tool names.

Defer platform protocols	No demonstrated reuse/delegation need yet.	MCP/A2A/GenUI not required in current implementation.

What we have deliberately not built

Future capability	Introduce when this requirement appears
MCP / tool governance	Several independent AI clients need interoperable, reusable governed tools.
RAG / runbook retrieval	Large, permissioned and changing document collections become necessary.
Durable orchestration / approvals	Long-running work or production writes must survive interruption and require approved action.
AG-UI, A2UI or MCP Apps	Specific frontend streaming, declarative rendering or embedded-host UI needs emerge.
Multi-agent / A2A	Independent specialist teams or services require delegated tasks with their own lifecycle.
Enterprise operations	Real users, tenants, model changes and traffic justify IAM, tracing, eval gates, quotas and recovery.

CHAPTER

Glossary and source map

- **p95 latency** — 95th percentile of request latency; 95% of measured requests are at or below this value.
- **Evidence ID** — Stable reference to a record retrieved from an approved source.
- **Grounding** — Supporting generated claims with retrieved evidence.
- **Authorization (AuthZ)** — Decision that a verified actor may perform a specified operation on a resource.
- **Tool allowlist** — Explicit set of permitted executable operations.
- **Tool budget** — Limit on number or cost of tool calls during one run.
- **Structured output** — Model response shaped according to a defined schema.
- **Trust boundary** — Point where input/data crosses into a system with different authority.
- **MCP** — Tool/data interoperability boundary between compatible AI host clients and servers.
- **A2A** — Agent-to-agent communication across independently operating services.
- **AG-UI** — Agent-to-frontend run/event protocol, distinct from MCP tool transport.
- **Idempotency** — Property that retrying the same logical action does not create duplicate effects; needed when writes arrive later.

Source materials

This consolidated guide is based on the earlier lessons through Stage 5 Evolution 1 and the flat project sources that defined the teaching context:

- `SKILL.md` — progressive teaching contract.
- `sdt-catalog.md` — design #73 route.
- `sdt-module-09-advanced-ai.md` — primary capstone module.
- `sdt-anchor-73-unified-enterprise-agentic-platform.md` — case-specific sequence and trade-offs.
- `concept-protocol-boundaries.md` — why MCP, A2A and UI protocols solve different needs.
- `concept-production-controls.md` — authorization, grounding, observability, and failure controls.

Official documentation for future implementation checks: OpenAI function calling · OpenAI structured outputs · MCP specification. Version-dependent SDK behavior should be checked again before production use.

> **Accuracy note** All incident figures are fictional. The Stage 4 and Stage 5A test counts are taken from earlier lesson reports, not re-run as part of generating this consolidated source. This document includes no credentials, complete Python code, or live API calls.

CHAPTER

Next planned increment (not included)

Stage 5 Evolution 2 — Shared tools and MCP. The next problem is tool reuse across several AI products. We would compare direct Python/API integrations with an approved MCP server and catalog, then derive compatibility, ownership, and authorization requirements before implementing the extra boundary.

No MCP server, agent-to-agent protocol, production approval workflow, or full unified platform has been implemented in the stages covered by this file.

Curriculum and Documentation Sources

This consolidated guide is based on the previous lesson's Stage 1–4 and Stage 5 Evolution 1 explanations, supplemented by the following **project reference files**:

- `SKILL.md` — progressive teaching, mock implementation, and safety rules.
- `sdt-catalog.md` — maps AI design #73 to Module 09 and anchor #73.
- `sdt-module-09-advanced-ai.md` — #73 platform case brief.
- `sdt-anchor-73-unified-enterprise-agentic-platform.md` — focused capstone scope, staged evolution, and design trade-offs.
- `concept-production-controls.md` — authorization, grounding, audit, retries, and evaluation principles.
- `concept-protocol-boundaries.md` — distinctions between MCP, MCP Apps, AG-UI, A2UI, A2A, and Skills.

Official documentation links used as references in the earlier Stage 5 lesson:

- OpenAI function calling: <https://developers.openai.com/api/docs/guides/function-calling>
- OpenAI structured outputs: <https://developers.openai.com/api/docs/guides/structured-outputs>

These are **reference URLs**, not a claim that the optional live-model adapter has been re-tested against a hosted service in this document-generation turn.

Document scope ends at Stage 5, Evolution 1.

Next — Stage 5 — Evolution 2: MCP

Continue to Stage 5